INFORME 3

Hecho por Manuel Muñoz y Samuel Martínez

1. Introducción

En este informe se aborda una evolución avanzada del problema clásico del productor-consumidor. Partiendo de una base de concurrencia inestable (carreras críticas), se ha implementado una arquitectura robusta de sincronización utilizando hilos POSIX (pthreads), exclusión mutua (mutex) y variables de condición.
El escenario principal consta de tres hilos productores independientes y un único hilo consumidor. Para resolver el problema de la prioridad sin provocar bloqueos innecesarios (deadlocks o esperas activas), el sistema se ha diseñado con tres colas FIFO independientes, una para cada nivel de prioridad. A lo largo de las distintas versiones del código, se introducen nuevas restricciones en el sistema, como la caducidad temporal de los ítems y la variación en las velocidades de producción y consumo.

2. Ejecución

Para compilar y ejecutar cualquiera de las versiones de este sistema, se asume la existencia de un Makefile con directivas similares a las de las prácticas anteriores (run y clean). La compilación general requiere vincular la librería de hilos: gcc -Wall -Wextra -o ejecutable archivo.c -lpthread.
La ejecución de todos los programas requiere pasar por argumento tres archivos de texto que contienen los enteros a procesar (uno para cada productor): ./ejecutable numeros1.txt numeros2.txt numeros3.txt.

3. Funcionamiento

El primer programa establece la arquitectura principal que heredarán el resto de los ejercicios.

- Productores (3). Cada uno lee de un fichero distinto. El Productor 1 genera ítems de prioridad 1 (la más alta), el 2 de prioridad 2, y el 3 de prioridad 3 (la más baja). Sus tiempos de producción son aleatorios entre 1 y 6 segundos.
- Gestión de colas. En lugar de un búfer único, se emplea un array de tres búferes circulares compartidos (comp[NUM_PROD]). Cada cola está protegida por su propio mutex y variable de condición. Esto permite que los tres productores inserten datos en paralelo sin bloquearse entre sí.
- Consumidor con trylock. El consumidor debe priorizar los elementos de mayor prioridad. Para evitar quedarse bloqueado esperando en la cola 1 si hay elementos disponibles en la cola 3, se emplea la función pthread_mutex_trylock. El consumidor itera sobre las colas en orden de prioridad (0, 1, 2); si el mutex está ocupado por un productor

insertando, el trylock falla sin bloquear el hilo, pasando inmediatamente a revisar la siguiente cola.
- Prevención de espera activa. Para evitar que el consumidor queme ciclos de CPU haciendo polling (revisando las colas continuamente cuando todas están vacías), se utiliza un contador global de ítems (items_totales_disponibles) protegido por un mutex global (mutc). El consumidor se duerme de forma eficiente hasta que un productor le avise mediante un signal de que hay trabajo disponible.

Las siguientes iteraciones del código mantienen la arquitectura base, pero añaden complejidad al ciclo de vida de los datos.

Inclusión de tiempos de caducidad (prod3_cons1_caduca.c)

En esta versión, los datos ya no son persistentes indefinidamente.
- Generación de Tiempos. Al insertar un ítem, el productor le asigna un tiempo_creacion exacto en milisegundos (usando gettimeofday) y un tiempo_caducidad aleatorio entre 1 y 12 segundos.
- Descarte en el consumidor. Al extraer el ítem, el consumidor calcula el tiempo transcurrido. Si tiempo_transcurrido > tiempo_caducidad, el ítem se considera caducado. El consumidor imprime un aviso por pantalla, ignora el valor (no lo suma al total) y omite el tiempo de simulación (sleep), pasando inmediatamente al siguiente ítem.

Tiempos de consumo fijos por prioridad (prod3_cons1_sleeps_a.c)

Este ejercicio modifica el comportamiento del consumidor de la versión anterior para simular que las tareas prioritarias son más costosas computacionalmente.
- El tiempo de procesamiento (simulado con sleep) deja de ser aleatorio.
- Se aplica la fórmula t = 4 - prioridad.
- Como resultado, procesar un ítem de prioridad 1 tarda 3 segundos, prioridad 2 tarda 2 segundos, y prioridad 3 tarda 1 segundo. Esto genera un cuello de botella observable cuando el consumidor encuentra muchos ítems de prioridad 1 seguidos.

Tiempos de producción fijos por prioridad (prod3_cons1_sleeps_b.c)

La última versión restaura el tiempo de consumo a un valor aleatorio (1-3s), pero modifica drásticamente las tasas de generación de los productores:
- Productor 1 (Prioridad 1). Inserta un ítem cada 7 segundos (generación muy lenta).
- Productor 2 (Prioridad 2). Inserta un ítem cada 5 segundos.
- Productor 3 (Prioridad 3). Inserta un ítem cada 1 segundo (generación muy rápida).
- Impacto en el sistema. El consumidor pasará la mayor parte de su tiempo procesando elementos de baja prioridad, ya que la cola 3 se llena constantemente. El uso de trylock brilla especialmente aquí, ya que el consumidor drena eficientemente la cola 3, interrumpiendo este flujo solo en los escasos momentos en los que la cola 1 o 2 reciben un nuevo ítem.

4. Resultados

prod3_cons1.c

Se observa cómo el consumidor prioriza los ítems de los Productores 1 y 2 cuando hay disponibilidad simultánea, y cómo calcula las sumas de forma independiente.

```
[2001] [CONS] Todos los buffers vacios. Esperando...
[4000] [PROD 2] Insertado '44' (Prioridad 2). Longitud: 1
[4000] [PROD 2] Sleep aleatorio (Iter 2). Esperando 6s...
[4000] [PROD 1] Insertado '13' (Prioridad 1). Longitud: 1
[4000] [PROD 1] Sleep aleatorio (Iter 2). Esperando 3s...
[4000] [CONS] Eliminado '13' (Prioridad 1). Longitud: 0
[4000] [CONS] Procesando '13' (Prioridad 1). Esperando 2s...
[6001] [CONS] Consumido '13' (Prioridad 1). Sumas acumuladas: 13 (PROD 1) 0 (PROD 2) 97 (PROD 3)
[6001] [CONS] Eliminado '44' (Prioridad 2). Longitud: 0
[6001] [CONS] Procesando '44' (Prioridad 2). Esperando 3s...
```

```
[PROD 1] Suma acumulada: 3902
[PROD 2] Suma acumulada: 4018
[PROD 3] Suma acumulada: 3944
[CONS] Sumas acumuladas: 3902 (PROD 1) 4018 (PROD 2) 3944 (PROD 3)
```

Como era de esperar, los ítems producidos se corresponden con los consumidos.

prod3_cons1_caduca.c

Se muestra un caso donde el consumidor extrae un ítem, pero este es descartado al haber superado su límite de vida, omitiendo la suma.

```
[263019] [PROD 2] Insertado '31'. Longitud: 1 | Caducidad: 2s
[263019] [PROD 2] Sleep aleatorio (Iter 78). Esperando 4s...
```

```
[269020] [CONS] Extraido '31' (Prioridad 2). Longitud: 1
[269020] [CONS] Item '31' (Prioridad 2) caducado. No se suma. Tiempo transcurrido: 6s
```

Al implementar la caducidad junto con la prioridad, en las sumas acumuladas se ve reflejado como los ítems de prioridad 3 son los que más se descartan; mientras que, los de prioridad 1 son los menos afectados.

```
[PROD 1] Suma acumulada: 3902
[PROD 2] Suma acumulada: 4018
[PROD 3] Suma acumulada: 3944
[CONS] Sumas acumuladas: 3789 (PROD 1) 2652 (PROD 2) 1233 (PROD 3)
```

prod3_cons1_sleeps_a. y prod3_cons1_sleeps_b.c

Se aprecia el alto volumen de inserciones del Productor 3 y cómo el consumidor atiende rápidamente esos ítems, saltando momentáneamente a los escasos ítems de prioridad 1 cuando estos aparecen cada 7 segundos (prod3_cons1_sleeps_b.c).

```
[131016] [CONS] Extraido '75' (Prioridad 3). Longitud: 4
[131016] [CONS] Item '75' (Prioridad 3) caducado. No se suma. Tiempo transcurrido: 11s
[131016] [CONS] Consumido '75' (Prioridad 3). Sumas acumuladas: 982 (PROD 1) 1319 (PROD 2) 937 (PROD 3)
[131016] [CONS] Extraido '4' (Prioridad 3). Longitud: 3
[131016] [CONS] Item '4' (Prioridad 3) caducado. No se suma. Tiempo transcurrido: 9s
[131016] [CONS] Consumido '4' (Prioridad 3). Sumas acumuladas: 982 (PROD 1) 1319 (PROD 2) 937 (PROD 3)
[131016] [CONS] Extraido '83' (Prioridad 3). Longitud: 2
[131016] [CONS] Item '83' (Prioridad 3) caducado. No se suma. Tiempo transcurrido: 8s
[131016] [CONS] Consumido '83' (Prioridad 3). Sumas acumuladas: 982 (PROD 1) 1319 (PROD 2) 937 (PROD 3)
[131016] [CONS] Extraido '82' (Prioridad 3). Longitud: 1
[131016] [CONS] Item '82' (Prioridad 3) caducado. No se suma. Tiempo transcurrido: 7s
[131016] [CONS] Consumido '82' (Prioridad 3). Sumas acumuladas: 982 (PROD 1) 1319 (PROD 2) 937 (PROD 3)
[131016] [CONS] Extraido '76' (Prioridad 3). Longitud: 0
[131016] [CONS] Procesando '76' (Prioridad 3). Esperando 1s...
[132016] [CONS] Consumido '76' (Prioridad 3). Sumas acumuladas: 982 (PROD 1) 1319 (PROD 2) 1013 (PROD 3)
[132017] [CONS] Todos los buffers vacios. Esperando...
[133004] [PROD 1] Insertado '14'. Longitud: 1 | Caducidad: 7s
[133004] [PROD 1] Sleep fijo (Iter 20). Esperando 7s...
[133004] [CONS] Extraido '14' (Prioridad 1). Longitud: 0
[133004] [CONS] Procesando '14' (Prioridad 1). Esperando 3s...
```

Cabe destacar los resultados obtenidos en las sumas acumuladas. En la primera implementación con los sleeps variables por prioridad en el consumidor (prod3_cons1_sleeps_a.c), el principal perjudicado es el productor intermedio (con sleeps de 2 segundos).

```
[PROD 1] Suma acumulada: 3902
[PROD 2] Suma acumulada: 4018
[PROD 3] Suma acumulada: 3944
[CONS] Sumas acumuladas: 3493 (PROD 1) 1914 (PROD 2) 2036 (PROD 3)
```

Sin embargo, para la segunda implementación (prod3_cons1_sleeps_b.c) con sleeps fijos por prioridad en el hilo productor, el productor 3, pese a dormirse solo por 1 segundo, se descartan gran cantidad de los ítems que produce. Si lo comparamos con la implementación anterior, el consumidor descarta menos ítems de los dos productores con más prioridad, sobre todo al segundo productor.

```
[PROD 1] Suma acumulada: 3902
[PROD 2] Suma acumulada: 4018
[PROD 3] Suma acumulada: 3944
[CONS] Sumas acumuladas: 3807 (PROD 1) 3928 (PROD 2) 1013 (PROD 3)
```

5. Conclusión

La evolución de estos ejercicios demuestra que el uso de estructuras de datos separadas (una cola por prioridad) simplifica enormemente la gestión de políticas complejas. El uso combinado de variables de condición globales (para evitar el consumo innecesario de CPU) junto con pthread_mutex_trylock (para la comprobación ágil de prioridades sin caer en interbloqueos) resulta ser una estrategia altamente efectiva para resolver el problema del productor-consumidor en entornos asimétricos. Adicionalmente, el registro de timestamps ha permitido implementar lógicas de tiempo real suave, como la caducidad de mensajes, manteniendo la coherencia y estabilidad de la memoria compartida.